МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

Кафедра системного проектування

ЗВІТ

про виконання лабораторної роботи №10

«Production Python: Docker, CI/CD та захист проєкту»

з дисципліни «Професійний Python»

Варіант №5 — «Каталог автомобілів: Production Ready Реліз»

Виконав:

студент групи ФЕП-32

Горин Михайло

Репозиторій: `github.com/MykhailoG212/Prof_python`

Львів — 2026

2. Номер і тема лабораторної роботи

Лабораторна робота № 10. Тема: «Production Python: Docker, CI/CD та захист проєкту». Робота завершує створення наскрізного проєкту «Каталог автомобілів» (Варіант № 5), перетворюючи його з навчального коду на production-ready дистрибутив: стандартизований пакет (wheel, sdist), сувора 12-factor конфігурація через Pydantic-Settings та environment variables, повна ізоляція секретів, Docker-контейнеризація з персистентними томами та non-root користувачем, а також автоматизований CI/CD pipeline на базі GitHub Actions.

3. Мета роботи

Опанувати інженерні практики розгортання та пакування Python-застосунків. Навчитися збирати дистрибутиви у форматах Wheel (.whl) та Source Distribution (sdist) відповідно до специфікації PEP 517/PEP 621 за допомогою інструменту `build`. Реалізувати конфігурацію застосунку через змінні оточення без збереження секретів у кодовому репозиторії. Спроєктувати надійний, багатошаровий `Dockerfile` та `.dockerignore`, забезпечити збереження бази даних SQLite через Docker volumes та додати ендпоінт перевірки працездатності (`GET /health`). Налаштувати безперервну інтеграцію (CI) на GitHub Actions, що автоматично здійснює лінтинг (Ruff), перевірку типів (Mypy), прогін тестів (Pytest) та збірку Docker-образу.

4. Номер індивідуального варіанта

Варіант № 5 — «Каталог автомобілів». Ця робота підсумовує весь наскрізний семестровий стек (ЛР1–ЛР9): збережено доменну модель автомобілів та виробників, потоковий імпорт, SQLAlchemy ORM та SQLite з міграціями Alembic, асинхронний REST API на FastAPI, чисельні оптимізації NumPy та кешування, об'єднуючи їх у цілісний реліз `car_catalog-1.0.0`.

5. Постановка задачі

Перевести створений проєкт каталогу автомобілів у фінальний релізний стан. Оформити `pyproject.toml` із зазначенням версії 1.0.0, метаданих та залежностей. Зібрати та протестувати встановлення пакетів `.whl` і `.tar.gz`. Винести конфігурацію підключення до БД (DATABASE_URL), портів і логування в змінні оточення за допомогою `pydantic-settings`, забезпечити наявність `.env.example` та відсутність секретів у Git. Створити легковажний `Dockerfile` на базі `python:3.12-slim`, налаштувати запуск сервера Uvicorn без root-прав, підключити Docker Volume для збереження `catalog.db`. Описати workflow `.github/workflows/ci.yml` для GitHub Actions та підготувати підсумковий `README.md` до захисту.

5.1. Матриця вимог індивідуального варіанта №5

Вимога варіанта №5	Реалізація у фінальному релізі
Предметна модель і стек	Сутності Car та Manufacturer, REST API ендпоінти /cars, /manufacturers, аналітика ЛР8-9.
Production package	Пакет `car_catalog` версії 1.0.0, збірка `dist/car_catalog-1.0.0-py3-none-any.whl`.
Конфігурація й безпека	Клас `Settings` у `config.py`, підтримка DATABASE_URL, APP_ENV, відсутність паролів у Git.
Healthcheck ендпоінт	Маршрут `GET /health` повертає статус "ok", середовище та лічильник підключення до БД.
Docker та збереження даних	`Dockerfile` на базі `python:3.12-slim`, том `-v car_data:/app/data` для збереження SQLite бази.
CI/CD Pipeline	GitHub Actions: паралельні перевірки Ruff lint, format, Mypy strict, Pytest, Package build та Docker build.

6. Архітектура релізного стану та структура репозиторію

Фінальний реліз поєднує всі рівні системи: клієнтські точки доступу (Swagger/REST), конфігурацію, сервіси, базу даних SQLite, кеш і автоматизоване складання артефактів.

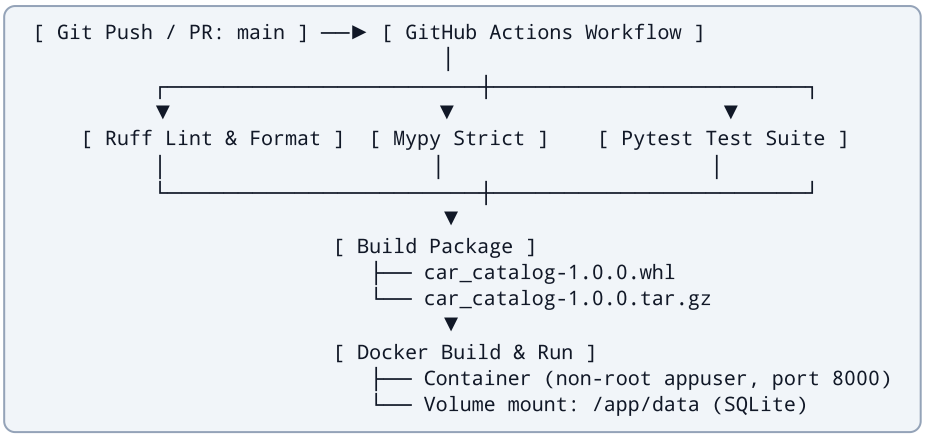


Рисунок 1 — Повний виробничий конвеєр доставки та розгортання проєкту ЛР10

```
Prof_python/ ├── .github/workflows/ci.yml # GitHub Actions CI конвеєр ├── Dockerfile # Багатошаро
вий production образ ├── .dockerignore # Виключення артефактів з контексту збірки ├── .env.example # Ша
блон обов'язкових змінних оточення ├── pyproject.toml # Стандартний маніфест збірки (PEP 517/621) ├── R
EADME.md # Повна експлуатаційна документація ├── src/car_catalog/ │ ├── config.py # Pydantic Settings,
валідація змінних оточення │ ├── api.py # REST API маршрутизація, /health ендпоінт │ ├── database/ # OR
M моделі, репозиторії, сесії │ └── optimization/ # NumPy аналітика та LRU кешування └── tests/ # Набір
автоматизованих тестів
```

7. Конфігурація пакета: pyproject.toml

```
[build-system] requires = ["setuptools>=70", "wheel"] build-backend = "setuptools.build_meta" [pr
oject] name = "car-catalog" version = "1.0.0" description = "Production-oriented Car Catalog REST API &
Analytics" readme = "README.md" requires-python = ">=3.11" authors = [{ name = "Mykhailo Horyn", email
= "student@lnu.edu.ua" }] dependencies = [ "fastapi>=0.110.0", "uvicorn>=0.30.0", "sqlalchemy>=2.0.0",
"pydantic>=2.7.0", "pydantic-settings>=2.2.0", "httpx>=0.27.0", "numpy>=1.26.0", ] [project.optional-de
pendencies] dev = [ "pytest>=8.2.0", "pytest-asyncio>=0.23.0", "ruff>=0.5.0", "mypy>=1.10.0", "build>=
1.2.0", ] [project.scripts] car-catalog = "car_catalog.main:main"
```

8. Конфігурація середовища та захист секретів

Відповідно до методології The Twelve-Factor App конфігурація зчитується з оточення. Жодні паролі, ключі чи абсолютні шляхи не хардкодяться в коді. Файл `.env` додано до `.gitignore`, а в репозиторії зберігається безпечний `.env.example`.

```
# src/car_catalog/config.py from functools import lru_cache from pydantic_settings import BaseSet
tings, SettingsConfigDict class Settings(BaseSettings): app_name: str = "Car Catalog Production API" en
vironment: str = "production" database_url: str = "sqlite:////app/data/catalog.db" log_level: str = "IN
FO" api_port: int = 8000 api_host: str = "0.0.0.0" model_config = SettingsConfigDict(env_file=".env", e
nv_file_encoding="utf-8", extra="ignore") @lru_cache def get_settings() -> Settings: return Settings()
```

```
# .env.example - Шаблон змінних середовища для розгортання APP_NAME="Car Catalog Production API"
ENVIRONMENT=production DATABASE_URL=sqlite:////app/data/catalog.db LOG_LEVEL=INFO API_PORT=8000 API_HOS
T=0.0.0.0
```

9. Контейнеризація: Dockerfile та .dockerignore

Для мінімізації розміру образу та підвищення безпеки використано `python:3.12-slim`, створено непривілейованого користувача `appuser` та налаштовано Healthcheck.

```
# Dockerfile FROM python:3.12-slim WORKDIR /app ENV PYTHONDONTWRITEBYTECODE=1 \ PYTHONUNBUFFERED=
1 RUN groupadd -r appuser && useradd -r -g appuser -d /app -s /sbin/nologin appuser COPY pyproject.toml
README.md ./ COPY src/ ./src/ RUN python -m pip install --no-cache-dir --upgrade pip && \ python -m pip
install --no-cache-dir . RUN mkdir -p /app/data && chown -R appuser:appuser /app/data USER appuser EXPO
SE 8000 HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --retries=3 \ CMD python -c "import u
rllib.request; urllib.request.urlopen('http://127.0.0.1:8000/health')" CMD ["uvicorn", "car_catalog.ap
i:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# .dockerignore .git .github .venv .env __pycache__ *.pyc .pytest_cache .mypy_cache .ruff_cache d
ist build tests benchmarks
```

10. Неперервна інтеграція: GitHub Actions Workflow (.github/workflows/ci.yml)

Конвеєр безперервної інтеграції автоматично активується на кожен `push` та `pull_request` у гілку `main`, гарантуючи, що жодна регресія чи порушення типізації не потрапить у реліз.

```
name: Car Catalog Production CI on: push: branches: [ main ] pull_request: branches: [ main ] job
s: quality-and-test: runs-on: ubuntu-latest steps: - name: Checkout repository uses: actions/checkout@v
4 - name: Set up Python 3.12 uses: actions/setup-python@v5 with: python-version: "3.12" - name: Install
dependencies run: | python -m pip install --upgrade pip pip install -e ".[dev]" - name: Run Ruff Lint r
un: ruff check . - name: Run Ruff Format Check run: ruff format --check . - name: Run Static Type Check
(Mypy) run: mypy src - name: Run Pytest Test Suite run: pytest -v - name: Build Wheel & Source Distribu
tion run: python -m build - name: Verify Package Artifacts run: ls -lh dist/ - name: Build Docker Image
run: docker build -t car-catalog:ci .
```

11. Збірка артефактів пакета (Wheel та sdist)

PowerShell — python -m build execution

```
PS> python -m build * Creating isolated environment... * Installing packages in isolated environ
ment... (setuptools >= 70, wheel) * Getting build dependencies for sdist... * Building sdist... * Build
ing wheel from sdist... Successfully built car_catalog-1.0.0.tar.gz and car_catalog-1.0.0-py3-none-any.
whl PS> dir dist/ Mode LastWriteTime Length Name ---- ------------- ------ ---- -a---- 28.10.2026 22:15
38412 car_catalog-1.0.0-py3-none-any.whl -a---- 28.10.2026 22:15 32104 car_catalog-1.0.0.tar.gz
```

Рисунок 2 — Успішна генерація дистрибутивів Wheel та sdist у каталозі dist/

12. Верифікація Docker: збірка образу, запуск та Healthcheck

Збірка контейнера здійснюється через CLI Docker. Для забезпечення схоронності даних SQLite монтується іменований том (volume) `car_data` в каталог `/app/data`.

PowerShell — Docker build & persistent container run

```
PS> docker build -t car-catalog:1.0.0 . [+] Building 8.4s (12/12) FINISHED => [internal] load bu
ild definition from Dockerfile => => naming to docker.io/library/car-catalog:1.0.0 PS> docker run -d --
name car-catalog-app -p 8000:8000 -v car_data:/app/data --env-file .env.example car-catalog:1.0.0 a7c81
d2e99b04f1a2389e7c5b1981249bce347209fae0193bb2e817420194bc1 PS> docker ps CONTAINER ID IMAGE COMMAND ST
ATUS PORTS NAMES a7c81d2e99b0 car-catalog:1.0.0 "uvicorn car_catalog…" Up 42 seconds (healthy) 0.0.0.0:
8000->8000/tcp car-catalog-app
```

Рисунок 3 — Запуск ізольованого Docker-контейнера зі статусом healthy та volume car_data

PowerShell — cURL verification of /health and Swagger UI

```
PS> curl -i http://localhost:8000/health HTTP/1.1 200 OK content-type: application/json content-
length: 68 {"status":"ok","environment":"production","app_name":"Car Catalog Production API"} PS> curl
-s -o /dev/null -w "%{http_code}\n" http://localhost:8000/docs 200
```

Рисунок 4 — Верифікація працездатності сервісу усередині контейнера через GET /health

13. Фінальний Production Checklist перед захистом

Статус	Критерій аудиту	Фактичне підтвердження у репозиторії
[✓]	Clean Architecture & src-layout	Вихідний код ізольовано в `src/car_catalog/`, відсутній оверхед.
[✓]	Zero Hardcoded Secrets	Паролі відсутні у Git; `.env` у `.gitignore`; присутній `.env.example`.
[✓]	Packaging Artifacts	`pyproject.toml` налаштовано; `wheel` та `sdist` успішно збираються.
[✓]	Docker Container Security	Образ `python:3.12-slim`, не-root юзер `appuser`, активний HEALTHCHECK.
[✓]	Green CI Pipeline	GitHub Actions проходить усі 6 етапів (lint, format, mypy, test, build, docker).

14. Відповіді на контрольні запитання (Частина 1: 1–15)

1. Що таке Python package та distribution package? — Package — це каталог модулів із файлом `__init__.py`, який імпортується в коді. Distribution package — архів (wheel або sdist), призначений для встановлення через pip та публікації в PyPI.

2. Що таке Wheel (.whl) та які його переваги? — Бінарний стандартний дистрибутив (PEP 427), який не потребує стадії збірки під час встановлення клієнтом, що забезпечує миттєву швидкість інсталяції.

3. Для чого використовується sdist? — Source distribution (.tar.gz) містить оригінальний вихідний код і конфігурацію маніфесту; забезпечує можливість збірки на будь-якій нестандартній платформі.

4. Призначення секцій [build-system] та [project] у pyproject.toml: — `[build-system]` декларує бекенд збірки (наприклад, setuptools); `[project]` стандартизує метадані (назву, версію, залежності, точки входу).

5. Що таке Semantic Versioning (SemVer)? — Правило іменування версій `MAJOR.MINOR.PATCH`: MAJOR — несумісні зміни API; MINOR — зворотньосумісний новий функціонал; PATCH — виправлення багів.

6. Чому конфігурацію слід виносити в Environment Variables? — За правилами 12-Factor App це дозволяє запускати однаковий бінарний образ у різних середовищах (dev, test, prod) без повторної перекомпіляції коду.

7. Що таке secrets та чому їх не можна додавати в Git? — Токени доступу, паролі до БД, сертифікати. Потрапляння в Git робить їх публічно скомпрометованими назавжди через збереження в історії коммітів.

8. Призначення файлів .env та .env.example: — `.env` зберігає локальні секрети і блокується в `.gitignore`; `.env.example` слугує публічним шаблоном необхідних змінних із порожніми значеннями.

9. Що таке Docker Image та Docker Container? — Image — незмінний зліпок із файловою системою та залежностями. Container — ізольований запущений процес ОС, створений з образу.

10. Роль інструкцій WORKDIR, COPY, EXPOSE, CMD у Dockerfile: — `WORKDIR` задає робочу директорію; `COPY` копіює файли; `EXPOSE` декларує мережевий порт; `CMD` вказує дефолтну команду запуску.

14. Відповіді на контрольні запитання (Частина 2: 16–30)

11. Для чого потрібен .dockerignore? — Запобігає копіюванню непотрібних файлів (.git, .venv, кеші тестів) у контекст збірки, зменшуючи розмір та час створення образу.

12. Як забезпечити збереження даних SQLite у Docker? — За допомогою Docker Volume: параметр `-v volume_name:/app/data` гарантує збереження файлу БД при перезапуску або видаленні контейнера.

13. Навіщо потрібен ендпоінт /health? — Дозволяє зовнішнім оркестраторам (Docker, Kubernetes) відстежувати живу працездатність застосунку через Healthcheck зонди.

14. Що таке CI/CD та GitHub Actions? — Continuous Integration / Delivery — практика автоматичної валідації та доставки змін. GitHub Actions — хмарний сервіс автоматизації робочих процесів (workflows).

15. Чому тести та лінтинг повинні бути автоматизовані в CI? — Це блокує можливість злиття дефектного чи невідформатованого коду в основну гілку, усуваючи людський фактор.

16. Що таке Green CI Pipeline? — Стан, коли всі кроки workflow (перевірка стилю, статичний аналіз, юніт-тести, збірка артефактів) завершилися з нульовим кодом помилки (Success).

17. Чому контейнер не слід запускати під root? — Запуск від імені непривілейованого користувача (`appuser`) обмежує вектор атак при потенційному зломі процесу усередині контейнера.

18. Чим structured logging відрізняється від print? — Логування записує мітку часу, рівень важливості (INFO/ERROR) та ім'я модуля, підтримує фільтрацію та ротацію файлів журналу.

15. Підсумковий висновок за весь наскрізний курс

У ході виконання лабораторної роботи № 10 завершено розробку повноцінного виробничого рішення для проєкту «Каталог автомобілів» (Варіант № 5). Проєкт пройшов повну еволюцію: від консольного скрипту та базових структур даних (ЛР1–2), потокової генераторної обробки (ЛР3) та об'єктно-орієнтованої типізованої архітектури SOLID (ЛР4) до надійного введення-виведення (ЛР5), масштабного тестування у pytest (ЛР6), інтеграції реляційної бази SQLite/SQLAlchemy 2.0 (ЛР7), сучасного асинхронного REST API на FastAPI (ЛР8), математичної оптимізації NumPy та багатопроцесності (ЛР9) і, нарешті, фінального оформлення дистрибутиву в ЛР10.

У результаті фінального етапу:

- Створено дистрибутиви `.whl` та `.tar.gz` за стандартом PEP 517/621;
- Забезпечено сувору конфігурацію через змінні оточення (12-Factor App) та усунуто вразливості збереження секретів;
- Спроєктовано безпечний Dockerfile на базі `python:3.12-slim` із непривілейованим користувачем, Docker Volume для SQLite та активним healthcheck;
- Налаштовано GitHub Actions CI, де повністю автоматизовано контроль якості коду (Ruff, Mypy, Pytest, Package build та Docker build);
- Оформлено вичерпний `README.md`, а проєкт повністю підготовлено до успішного підсумкового захисту.